

ID	Title	Method	Request Parameters	Test Objective	Test Steps	Expected Result	Actual Result	Status
CREATE								
TC_01	Create card – valid data	POST	idList={{listId}} name=API CARD key={{APIKey}} token={{APIToken}}	Verify a card is created with all valid parameters.	1. Set method to POST 2. Add idList, name, key, token 3. Click Send	200 OK – response body contains card object with correct name and idList.	200 OK – Card created with correct name.	Pass
TC_02	Create card – missing idList	POST	name=API CARD key={{APIKey}} token={{APIToken}} (idList omitted)	Verify the API rejects the request when idList is missing.	1. Set method to POST 2. Remove idList param 3. Click Send	400 Bad Request – error: "invalid value for idList"	400 Bad Request – "invalid value for idList"	Pass
TC_03	Create card – invalid token	POST	idList={{listId}} name=API CARD key={{APIKey}} token=INVALID_STRING	Verify the API returns 401 when token is wrong.	1. Set method to POST 2. Replace token with random invalid string 3. Click Send	401 Unauthorized – message: "invalid app token"	401 Unauthorized – "invalid app token"	Pass
TC_04	Create card – empty name	POST	idList={{listId}} name= (empty) key={{APIKey}} token={{APIToken}}	Verify the API allows creating a card with an empty name (defaults to null).	1. Set method to POST 2. Leave name value blank 3. Click Send	200 OK – card created, name field is null in response.	200 OK – Card created, name: null	Pass
TC_05	Create card – special characters in name	POST	idList={{listId}} name= & "Card!" key={{APIKey}} token={{APIToken}}	Verify the API handles special characters in name without errors.	1. Set method to POST 2. Set name with special chars: & "Card"! 3. Click Send	200 OK – card created, name stored/encoded correctly.	200 OK – Card created with special char name.	Pass
TC_06	Create card – max length name	POST	idList={{listId}} name=(16,384-char string) key={{APIKey}} token={{APIToken}}	Verify API behavior when name exceeds character limit.	1. Set method to POST 2. Generate name string of 16,384+ chars 3. Click Send	400 Bad Request – error indicating name exceeds max length.	400 Bad Request – max length error returned.	Pass
TC_07	Create card – missing API key	POST	idList={{listId}} name=API CARD token={{APIToken}} (key omitted)	Verify the API returns 401 when the API key is missing.	1. Set method to POST 2. Remove key param 3. Click Send	401 Unauthorized – message: "invalid key"	401 Unauthorized – "invalid key"	Pass
READ (GET)								
TC_08	Get card – valid ID	GET	key={{APIKey}} token={{APIToken}}	Verify the API returns correct card details for an existing card.	1. Set method to GET 2. URL: { {base_url}}/cards/{ {CardID}}?key={{APIKey}}&token={{APIToken}} 3. Click Send	200 OK – response contains matching card ID, name, and idList.	200 OK – Correct card details returned.	Pass
TC_09	Get card – invalid ID format	GET	key={{APIKey}} token={{APIToken}} cardId=InvalidID123	Verify the API returns 400 when card ID format is invalid.	1. Set method to GET 2. Replace {{CardID}} with 'InvalidID123' 3. Click Send	400 Bad Request – error: "invalid id"	400 Bad Request – "invalid id"	Pass
TC_10	Get card – missing API key	GET	token={{APIToken}} (key omitted)	Verify the API blocks access when the API key is missing.	1. Set method to GET 2. Remove key from query params 3. Click Send	401 Unauthorized – message: "invalid key"	401 Unauthorized – "invalid key"	Pass
TC_11	Get card – invalid token	GET	key={{APIKey}} token=INVALID_STRING	Verify the API blocks access when an incorrect token is provided.	1. Set method to GET 2. Replace token with invalid string 3. Click Send	401 Unauthorized – message: "invalid app token"	401 Unauthorized – "invalid app token"	Pass

ID	Title	Method	Request Parameters	Test Objective	Test Steps	Expected Result	Actual Result	Status
UPDATE (PUT)								
TC_12	Update card name	PUT	key={{APIKey}} token={{APIToken}} name=Grad	Verify the card name can be successfully updated.	1. Set method to PUT 2. Add param: name=Grad 3. Click Send	200 OK – response body contains updated name 'Grad'.	200 OK – Card name changed to 'Grad'.	Pass
TC_13	Update card description	PUT	key={{APIKey}} token={{APIToken}} name=Grad desc=This is my Grad Project description	Verify the card description can be updated successfully.	1. Set method to PUT 2. Add params: name=Grad, desc=... 3. Click Send	200 OK – response body shows updated desc field.	200 OK – Description updated in response.	Pass
TC_14	Update card – empty name	PUT	key={{APIKey}} token={{APIToken}} name= (empty) desc=This is my Grad Project description	Verify how the API handles an update where the name is empty.	1. Set method to PUT 2. Set name= (empty), add desc 3. Click Send	200 OK – name remains unchanged or is set to empty/null.	200 OK – Card name unchanged or set to empty.	Pass
TC_15	Update card – invalid card ID	PUT	cardId=INVALIDID key={{APIKey}} token={{APIToken}} name=Test	Verify error handling when updating a card with an invalid ID.	1. Set method to PUT 2. Replace {{CardID}} with 'INVALIDID' 3. Click Send	400 Bad Request – error: "invalid id"	400 Bad Request – "invalid id"	Pass
TC_16	Update card – invalid token	PUT	key={{APIKey}} token=INVALID_STRING name=Grad	Verify the API returns 401 when an invalid token is used during update.	1. Set method to PUT 2. Use an invalid token string 3. Click Send	401 Unauthorized – message: "invalid app token"	401 Unauthorized – "invalid app token"	Pass
TC_17	Update card – missing API key	PUT	token={{APIToken}} name=Grad (key omitted)	Verify the API blocks update requests when the API key is missing.	1. Set method to PUT 2. Remove the key param 3. Click Send	401 Unauthorized – message: "invalid key"	401 Unauthorized – "invalid key"	Pass
DELETE								
TC_18	Delete existing card	DELETE	key={{APIKey}} token={{APIToken}}	Verify a card can be permanently deleted.	1. Set method to DELETE 2. URL: {{base_url}}/cards/{{CardID}}?key=...&token=... 3. Click Send	200 OK – response body: {"limits": {}} confirming deletion.	200 OK – Limits object returned, card deleted.	Pass
TC_19	Verify deletion – GET deleted card	GET	DeletedCardID (from TC_18) key={{APIKey}} token={{APIToken}}	Confirm the deleted card is no longer accessible.	1. Set method to GET 2. Use same CardID deleted in TC_18 3. Click Send	404 Not Found – message: "The requested resource was not found"	404 Not Found as expected.	Pass
TC_20	Delete non-existent card	DELETE	cardId=69fakeID123 key={{APIKey}} token={{APIToken}}	Verify error handling when deleting a card that does not exist.	1. Set method to DELETE 2. Use a fake/random card ID 3. Click Send	400 Bad Request – error: "invalid id"	400 Bad Request – "invalid id"	Pass
TC_21	Delete card – missing API key	DELETE	token={{APIToken}} (key omitted)	Verify the API blocks delete requests when API key is missing.	1. Set method to DELETE 2. Remove key param 3. Click Send	401 Unauthorized – message: "invalid key"	401 Unauthorized – "invalid key"	Pass
ADDITIONAL								

ID	Title	Method	Request Parameters	Test Objective	Test Steps	Expected Result	Actual Result	Status
TC_22	Create card – duplicate name in same list	POST	idList={{listId}} name=API CARD (same as TC_01) key={{APIKey}} token={{APIToken}}	Verify the API allows two cards with the same name in the same list.	1. Set method to POST 2. Use same name and idList as TC_01 3. Click Send	200 OK – second card created with a different unique ID.	200 OK – Duplicate card created with new ID.	Pass
TC_23	Move card to a different list	PUT	key={{APIKey}} token={{APIToken}} idList={{AnotherListId}}	Verify a card can be moved between lists by updating idList.	1. Set method to PUT 2. Add param: idList={{AnotherListId}} 3. Click Send	200 OK – response shows updated idList matching target list.	200 OK – Card moved, idList updated.	Pass
TC_24	Rate limiting – rapid repeated GET requests	GET	key={{APIKey}} token={{APIToken}}	Verify the API returns 429 when called excessively in a short window.	1. Set method to GET 2. Send 300+ requests in 10s via Postman Runner 3. Observe response codes	429 Too Many Requests – after hitting rate limit threshold.	429 Too Many Requests – limit triggered.	Pass